

Hackathon Challenge

Catch the Invisible AI Cheater

Track: Interview Integrity · Hosted by: InCruiter

The Challenge

AI "interview copilots" like **Cluely**, **Parakeet**, **FinalRound AI** etc. like many other tools let a candidate cheat through a live remote interview. They listen to the interviewer's question and silently feed the candidate an answer on a floating overlay of a teleprompter the interviewer can never see. These tools are built to be **invisible to screen sharing and screen recording**, so traditional proctoring sees nothing.

Your mission: build a way to **detect that a candidate is using one of these hidden overlay tools** during an interview.

The Twist (the hard part)

Two rules make this genuinely difficult and that's the point:

1. **Zero install.** You may **not** install any application, agent, or browser extension on the candidate's machine. Whatever you build has to work from the interview experience itself (a normal web page / interview session).
2. **Beat stealth mode.** These tools have a "private / stealth mode" that hides them from screen share, the taskbar, and recordings. Your solution should aim to catch them **even when stealth mode is on**.

Think creatively. The obvious approaches (look at the shared screen, check the task manager) are exactly what these tools are designed to defeat.

What You Build

A working prototype that, during a mock interview, can **raise a flag** when a hidden AI overlay is in use and present that flag to a reviewer in a clear, trustworthy way.

A strong submission will: - **Detect** as many real cheating scenarios as possible from a zero-install setup. - **Score the risk** (e.g. clean / suspicious / high-risk) instead of a blind yes/no. - **Show the evidence** to a human reviewer — a simple dashboard or report, not a black box. - **Avoid false alarms** on honest candidates (a normal person on a normal laptop must not get flagged). - **Be honest** about what it can and can't catch.

You do **not** need to perfectly catch every tool. We're looking for clever, working, defensible detection — and a clear-eyed view of the limits.

Ground Rules

- **No software, agent, or extension installed on the candidate's device.**
- **Never auto-reject a candidate.** Your output is a signal for a human reviewer, not an automatic verdict.
- **Respect privacy.** Get clear consent for anything you capture (camera, screen, audio). Don't store more than you need.
- Use any language, framework, or AI model you like.
- Build on open-source freely; cite what you use.

What to Submit

1. **A working demo** — runnable in one command, that flags at least one realistic cheating scenario live.
2. **A short write-up** (1 page) — what you built, how it detects, and an honest note on coverage and limits.
3. **A 3–5 minute presentation** — problem, your approach, live demo, what's next.
4. **A Git code URL**

How You're Judged (100 points)

Criteria

Points

What we look for

Detection that works :35

Actually catches cheating in a live demo; signals fire correctly

Creativity: 20

A smart, non-obvious approach to a deliberately hard problem

Low false alarms: 15

Honest candidates aren't flagged

Reviewer experience: 15

Clear evidence, risk score, and a usable dashboard/report

Honesty & rigor: 15

Realistic about coverage; no overclaiming

Heads up: claiming you "100% detect Cluely/Parakeet/FinalRound and similar tools" without proof will lose points. We value an honest, working partial solution over an impressive-sounding fake one.

Suggested Timeline (48 hours)

Time Phase:

Hour 0–2

Kickoff, form teams, understand the problem

Hour 2–10

Set up the interview session + capture basics

Hour 10–22

Build your detection signals

Hour 22–30

Risk scoring + reviewer dashboard/report

Hour 30–34

Test against honest users and cheating scenarios; reduce false alarms

Hour 34–36

Polish, prepare demo and slides

Final

Demos & judging

Ideas to Get You Started

You're free to ignore these they're just sparks:

- What can a normal web page legitimately *see* about the candidate's setup (with consent)?
- Cheating tools have to listen and answer. Does that leave any trace, timing pattern, or behavior?
- If a tool refuses to show itself, is the *refusal* or *evasion* itself a signal?
- Could you analyze what the **interviewer** sees, instead of the candidate's machine?
- How do you tell a real cheater apart from an honest person who just has a second monitor?

Eligibility & Teams

- Single Handedly should have built the solution.
- Bring your own laptop.
- All work must be created during the event; pre-existing open-source libraries are fine.

*Questions during the event? Connect with the Coordinator. **Good luck and have fun outsmarting the "undetectable."***